

Нейронные сети и глубокое обучение

Полный теоретический конспект

Темы 1–6 | Подготовка к экзамену

Тема 1. Введение в искусственные нейронные сети (ИНС)

1.1 Биологические основы

Искусственные нейронные сети инспирированы устройством биологической нервной системы. Основная тенденция развития кибернетики начала XXI века — биологизация (имитация механизмов живой природы) и гибридизация (совместное применение разных методов).

Биологический нейрон состоит из:

- Сомы (тела) — обрабатывает поступающие сигналы
- Дендритов — принимают входные сигналы
- Аксона — передаёт выходной сигнал другим нейронам
- Синапсов — места контакта аксона с дендритами/сомой других нейронов

Синаптическая сила аналогична весовым коэффициентам искусственного нейрона.

1.2 Формальный нейрон (модель МакКаллока–Питтса)

ИНС — параметрически заданная функция, отображающая входные сигналы в выходные с помощью модели из связанных формальных нейронов.

Стандартный формальный нейрон состоит из трёх элементов:

- **1. Адаптивный сумматор** — вычисляет взвешенную сумму входов $s = w_0 + \sum w_i x_i$ (скалярное произведение весов и входов). w_0 — смещение (bias). Если $w_0 = 0$ — однородный; если $w_0 \neq 0$ — неоднородный сумматор.
- **2. Нелинейная активационная функция** — вычисляет $y = f(s)$. Требования: монотонность, существование производной почти всюду, удобное выражение производной через саму функцию. Выпуклость не требуется.
- **3. Точка ветвления** — рассылает выходной сигнал y другим нейронам (аналог аксона).

💡 Весовые коэффициенты w_i — аналог синаптической силы биологической связи. «Адаптивным» сумматор называется потому, что его параметры настраиваются в процессе обучения.

1.3 Функции активации

Нелинейность активационной функции принципиально важна: суперпозиция линейных слоёв снова линейна, без нелинейности глубокая сеть эквивалентна одному линейному слою.

Сигмоида (функция Ферми)

$$f(s) = 1 / (1 + e^{-s})$$

Производная: $f'(s) = f(s) \cdot (1 - f(s))$. Значения в (0, 1). Проблема: производная мала (≤ 0.25), что приводит к затуханию градиента в глубоких сетях.

Гиперболический тангенс (tanh)

$$f(s) = \tanh(s) = (e^s - e^{-s}) / (e^s + e^{-s})$$

Производная: $f'(s) = 1 - \tanh^2(s)$. Значения в (-1, 1), центрирована вокруг нуля. Используется в РНС.

ReLU (Rectified Linear Unit)

$$f(s) = \max(0, s)$$

Производная: 1 при $s > 0$, 0 при $s \leq 0$. Основная активация в современных сетях. Проблема: «мёртвые» нейроны при $s < 0$.

Leaky ReLU

$$f(s) = s, \text{ если } s > 0; 0.01 \cdot s, \text{ если } s \leq 0$$

Отличие от ReLU: при $s < 0$ даёт малый ненулевой наклон ($0.01 \cdot s$), что снижает проблему «мёртвых» нейронов.

1.4 Архитектуры ИНС

Слоистые (многослойные) сети

Нейроны объединены в слои; сигнал проходит от входного слоя через скрытые к выходному. В трёхслойной сети: входной → скрытый → выходной слой.

Сигнал j -го нейрона слоя n : $S_j = \sum w_{ij} \cdot u_i$, затем $y_j = f(S_j)$. Нулевой «слой» — это вход: $u^0_j = x_j$.

Полносвязные (рекуррентные) сети

Каждый нейрон передаёт сигнал всем остальным, включая себя. Работают «тактами»; выходные сигналы — значения нейронов после нескольких тактов или в стационарном состоянии. Такт — прохождение сигнала через один слой. При неограниченном числе тактов слоистая сеть не работает, полносвязная — да.

Однородные и гетерогенные сети

- Однородная (гомогенная) — одинаковая функция активации во всех нейронах
- Гетерогенная (неоднородная) — активация зависит от параметров конкретного нейрона

1.5 Теоремы об аппроксимирующих возможностях ИНС

Теорема Колмогорова–Арнольда (1957)

Любую непрерывную функцию n переменных можно представить как суперпозицию функций одной переменной:

$$f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} h_q[\sum_{p=1}^n \varphi_p(x_p)]$$

Для двух переменных: $f(x,y) = \sum_{q=1}^5 h_q[\varphi_q(x) + \psi_q(y)]$. Внутренние функции φ стандартны (не зависят от f); от конкретной функции зависят лишь внешние h . Это обоснование 3-слойной архитектуры ИНС.

Теорема Вейерштрасса (1885)

Множество многочленов плотно в пространстве непрерывных функций на компакте: любую непрерывную функцию на замкнутом ограниченном множестве можно равномерно приблизить многочленами.

Компакт — замкнутое и ограниченное множество. Равномерная норма — максимум модуля отклонения (гарантирует близость в каждой точке).

Теорема Стоуна (1935)

Класс функций, замкнутый относительно алгебраических операций, содержащий единицу и разделяющий точки — плотен в $C(X)$. ИНС удовлетворяет этим условиям:

- Замкнутость относительно умножения реализуется через квадраты: $F_1 F_2 = \frac{1}{2}((F_1 + F_2)^2 - F_1^2 - F_2^2)$
- Единичная нейросеть: один нейрон с единичным весом, нулевым смещением, тождественной активацией
- Разделение точек: для любых $x \neq y$ значения функции различны

💡 Функция «разделяет точки»: для любых различных x и y значения $f(x) \neq f(y)$.

1.6 Перцептрон

Перцептрон — фактически один нейрон с пороговой функцией активации (± 1 в зависимости от знака суммы). Разработан Маккалоком и Питтсом в 1940-е гг.; основные результаты — Розенблатт в 1960-е гг. («Принципы нейродинамики», 1965).

Теорема сходимости Розенблатта: если задача линейно разделима, перцептрон обязательно обучится её решать.

Тема 2. Алгоритм обратного распространения ошибки

2.1 Суть метода

Backpropagation (обратное распространение ошибки) — алгоритм вычисления градиентов функции потерь по всем весам сети. Относится к классу алгоритмов стохастического градиентного спуска, применяется для слоистых ИНС. В основе лежит цепное правило дифференцирования сложной функции (chain rule, 1673 г.).

2.2 Прямой и обратный проход

Прямой проход (Forward pass)

На вход подаётся образец x , рассчитываются выходы нейронов от 1-го слоя до последнего:

$$S_j^n = \sum w_{ij} \cdot y_i^{n-1}, \quad y_j^n = f(S_j^n)$$

Здесь n — номер слоя, j — номер нейрона. После прямого прохода вычисляется ошибка на выходном слое.

Обратный проход (Backward pass)

Ошибки «дельта» распространяются от выходного слоя N к первому. Для выходного слоя ($n = N$):

$$g_j^N = y_j \cdot (1 - y_j) \cdot (d_j - y_j)$$

Здесь d_j — требуемый (эталонный) выход, y_j — фактический выход, $y_j \cdot (1 - y_j)$ — производная логистической функции.

Для скрытых слоёв ($n < N$) — взвешенная сумма ошибок следующего слоя:

$$g_j^n = y_j \cdot (1 - y_j) \cdot \sum_i g_i^{n+1} \cdot w_{ji}^{n+1}$$

g_j^n — чувствительность потерь к выходу сумматора нейрона (градиент потерь по S_j^n).

2.3 Корректировка весов

$$w_{ij} \leftarrow w_{ij} + \lambda \cdot g_j \cdot y_i$$

λ — скорость обучения (learning rate). Поправка пропорциональна скорости обучения, ошибке g_j и выходу предыдущего слоя y_i . Направление — антиградиент: движение в сторону убывания потерь ($w \leftarrow w - \lambda \cdot \partial L / \partial w$).

2.4 Алгоритм backprop (шаги)

- Шаг 1: инициализировать веса случайными числами из $[-1, 1]$
- Шаг 2: предъявить образец в случайном порядке (случайный порядок — чтобы сеть не «забывала» часть выборки)
- Шаг 3: прямой проход — вычислить выходы всех нейронов
- Шаг 4: вычислить ошибку на выходе
- Шаг 5: обратный проход — вычислить g для всех нейронов
- Шаг 6: скорректировать веса
- Шаг 7: проверить условие остановки; если ошибка значима — вернуться к шагу 2

Одна эпоха — полный проход по всей обучающей выборке (как правило, в случайном порядке).

2.5 Производные функций активации

- **Сигмоида:** $f'(s) = f(s) \cdot (1 - f(s))$ — производная выражается через значение самой функции (уже вычислено при прямом проходе, повторно считать не нужно)
- **tanh:** $f'(s) = 1 - \tanh^2(s)$
- **ReLU:** $f'(s) = 1$ при $s > 0$, иначе 0

Вычислительная эффективность backprop — переиспользование промежуточных производных. За один обратный проход считаются градиенты по всем весам (сложность линейная по числу связей), а не независимо для каждого веса.

2.6 Особые комбинации (критерий + активация)

Для softmax + кросс-энтропия: производная по аргументу для верного класса = $y_k - 1$, для остальных = y_k . Это сильно упрощает backprop.

Градиент квадратичной функции потерь $L = \frac{1}{2}(d-y)^2$ по выходу y равен $-(d-y)$.

2.7 Трудности backprop

Затухание градиента (Vanishing gradient)

Сигмоида имеет максимальную производную 0.25. При умножении по многим слоям градиент экспоненциально затухает. Решения: ReLU, нормализация батчей, skip-соединения.

Взрыв градиента (Exploding gradient)

При больших весах или некоторых архитектурах градиент экспоненциально растёт. Решения: gradient clipping, правильная инициализация.

Локальные минимумы и плато

Поверхность потерь невыпукла, градиентный спуск может застрять в локальном минимуме или плато. Стохастичность SGD помогает выходить из «неудачных» точек.

Симметрия весов

Нельзя инициализировать все веса нулями: все нейроны получают одинаковые градиенты и останутся дубликатами. Веса инициализируют случайно.

Тема 3. Обучение ИНС и глубоких нейронных сетей

3.1 Отличие МО от чистой оптимизации

В оптимизации цель — минимум функции. В машинном обучении минимизация эмпирического риска на обучающей выборке — лишь средство для хорошего обобщения на новых данных.

Идеальный критерий (математическое ожидание по истинному распределению) недоступен, поэтому используют эмпирический критерий — среднее по обучающей выборке.

3.2 Функции потерь

- **MSE (среднеквадратичная ошибка):** $L = (1/N) \cdot \sum (d_i - y_i)^2$ — для регрессии
- **Кросс-энтропия (CE):** $L = -\sum p \cdot \log(y)$ — для классификации, когда выход — вероятности
- **Binary CE:** специальный случай для бинарной классификации

Цель обучения ГНС — точечные (не интервальные) оценки весов. Хорошая оценка: состоятельная (сходится по вероятности), несмещённая (мат. ожидание = истинное значение), эффективная (минимальная дисперсия).

3.3 Алгоритмы оптимизации

SGD — стохастический градиентный спуск

Базовый метод. Градиент вычисляется по одному образцу (SGD) или мини-батчу (Mini-batch GD, BGD). Мини-батчи дают достаточно точную оценку при умеренных затратах и добавляют полезный стохастический шум.

$$w \leftarrow w - \lambda \cdot \nabla L(w)$$

Momentum (импульсный/моментный метод)

Добавляет инерцию: накапливает прошлые градиенты. Сглаживает траекторию и ускоряет сходимость в оврагах (узких долинах потерь), гася поперечные колебания.

$$v \leftarrow \beta \cdot v - \lambda \cdot \nabla L; \quad w \leftarrow w + v$$

Нестеров (NAG — Nesterov Accelerated Gradient)

Отличие от Momentum: градиент берётся в «упреждённой» точке (после применения момента), что даёт более точную коррекцию.

AdaGrad

Масштабирует шаг индивидуально для каждого параметра, деля на корень из накопленной суммы квадратов его градиентов. Редкие признаки получают больший шаг. Проблема: шаг затухает к нулю.

RMSPProp

Решает проблему AdaGrad: использует экспоненциальное скользящее среднее квадратов градиентов вместо накопленной суммы. Шаг не затухает к нулю.

Adam (Adaptive Moment Estimation)

Объединяет Momentum (1-й момент) и RMSProp (2-й момент) с коррекцией смещения.

$$m \leftarrow \beta_1 \cdot m + (1 - \beta_1) \cdot g; \quad v \leftarrow \beta_2 \cdot v + (1 - \beta_2) \cdot g^2 \\ \hat{m} = m / (1 - \beta_1^t); \quad \hat{v} = v / (1 - \beta_2^t); \quad w \leftarrow w - \lambda \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$$

Коррекция на $(1 - \beta^t)$ устраняет смещение оценок к нулю в начале обучения.

AdamW

Отделяет регуляризацию весов (decoupled weight decay) от адаптивного шага, что улучшает обобщение.

Методы второго порядка (Ньютона)

Используют Гессиан (вторые производные). В ГНС применяются редко: Гессиан имеет размер $(\text{число параметров})^2$ — вычисление и обращение неподъемны для миллионов параметров.

3.4 Регуляризация

L2-регуляризация (Ridge, Тихонов, weight decay)

Добавляет штраф $\frac{1}{2}\alpha \cdot \sum w_i^2$ к функции потерь. Даёт слагаемое αw в градиенте — каждый шаг веса «стягиваются» к нулю.

L1-регуляризация (Lasso)

Штраф $\alpha \cdot \sum |w_i|$. Стимулирует разреженность: многие веса становятся точно нулевыми (отбор признаков).

Dropout (прореживание)

На каждом шаге обучения случайно «выключает» часть нейронов (обнуляет их выход с вероятностью p). Типичные значения: входной слой $p \approx 0.2$, скрытые $p \approx 0.5$.

Регуляризирующий эффект: усреднение результатов множества случайных «подсетей».

Применяется только при обучении. При инференсе выходы масштабируются на $(1 - p)$ (или используют inverted dropout при обучении).

SpatialDropout2D — отключает целые карты признаков в СНС (вместо поточечного dropout).

Batch Normalization (пакетная нормализация)

Нормирует выходы слоя по среднему и дисперсии мини-батча, затем применяет обучаемые параметры γ (масштаб) и β (сдвиг). Ускоряет и стабилизирует обучение.

Layer Normalization

Вычисляет статистики по признакам одного объекта (независимо от батча). Удобна для РНС и трансформеров.

Ранняя остановка (Early Stopping)

Прекращает обучение при росте/застое валидационной ошибки. Предотвращает переобучение.

Аугментация данных

Расширение обучающей выборки допустимыми преобразованиями (повороты, отражения, шум и др.). Повышает устойчивость, снижает переобучение.

3.5 Инициализация весов

Цель: сохранить масштаб активаций и градиентов по слоям (предотвратить затухание/взрыв).

- **Инициализация Ксавье (Glorot):** дисперсия $\sim 2/(\text{fan_in} + \text{fan_out})$. Подходит для сигмоиды и $\tanh$.
- **Инициализация Хе (Kaiming):** дисперсия $\sim 2/\text{fan_in}$. Подходит для ReLU.
- **Смещения (bias):** обычно инициализируют нулями.

💡 Нельзя инициализировать все веса одинаково — нейроны останутся симметричными и не смогут обучиться разным признакам.

3.6 Расписание скорости обучения

- **Reduce on Plateau (RoP):** снижает скорость, когда метрика перестаёт улучшаться (выходит на плато)
- **Gradual Warmup:** постепенный рост скорости в начале обучения. Стабилизирует ранние этапы.
- **Cosine Decay:** плавное снижение скорости по косинусоиде. Даёт крупные шаги в начале и аккуратное «дошлифовывание» в конце.
- **Warm Restarts:** периодически повышают скорость, давая оптимизатору шанс выйти из узких минимумов.

3.7 Метрики качества

- **Accuracy:** доля верных предсказаний. При дисбалансе классов может вводить в заблуждение.
- **Precision:** $TP/(TP+FP)$ — доля верных среди предсказанных положительных
- **Recall (полнота):** $TP/(TP+FN)$ — доля найденных из всех истинных положительных
- **F1:** гармоническое среднее P и R: $F1 = 2PR/(P+R)$
- **При дисбалансе классов:** взвешивание классов в функции потерь, oversampling/undersampling

3.8 Переобучение и недообучение

- **Переобучение (overfitting):** малая train-ошибка, большая test-ошибка. Модель «запомнила» выборку. Лечение: регуляризация, dropout, ранняя остановка, больше данных.
- **Недообучение (underfitting):** большая ошибка уже на обучающей выборке. Модель недостаточно выразительна. Лечение: усложнение модели, более долгое обучение.

Тема 4. Свёрточные нейронные сети (СНС / CNN)

4.1 Мотивация и идея

В задачах обработки изображений полносвязные слои имеют огромное число параметров (каждый пиксель связан с каждым нейроном). СНС используют три ключевые идеи:

- Локальная связность — нейрон связан только с небольшой областью входа
- Разделение весов — один и тот же фильтр применяется ко всем позициям
- Пространственная инвариантность — одни и те же признаки могут встречаться в разных местах

Это резко снижает число параметров при сохранении числа нейронных связей.

4.2 Тензоры и операция свёртки

В СНС работают преимущественно с тензорами ранга 3: (высота × ширина × число каналов). Цветное изображение $H \times W \times 3$ — тензор ранга 3.

Перекры́стная корреляция (называемая «свёрткой»)

В СНС «свёрткой» формально называют перекры́стную корреляцию — ядро не переворачивается, как в классической свёртке:

$$s(n_1, n_2) = \sum_{\{m_1, m_2, k\}} w(m_1, m_2, k) \cdot x(n_1 + m_1, n_2 + m_2, k)$$

Суммирование по пространственным позициям m_1, m_2 и каналам k . По канальной координате суммирование без сдвига (глубина ядра = глубина входа).

Карта признаков (feature map)

Результат свёртки с одним ядром — двумерный тензор, карта признаков. L фильтров $\rightarrow L$ карт признаков на выходе.

Размеры выхода

- **Valid-свёртка (без padding):** выход $(N_1 - M_1 + 1) \times (N_2 - M_2 + 1)$. Размер уменьшается.
- **С дополнением P:** $(N_1 + P_1 - M_1 + 1) \times (N_2 + P_2 - M_2 + 1)$
- **Конгруэ́нтная свёртка (same):** размеры входа и выхода равны. Достигается подбором padding.

Число параметров фильтра

$$\text{Параметры} = M_1 \cdot M_2 \cdot K + 1 \text{ (} K \text{ каналов входа} + 1 \text{ смещение)}$$

Свёртка 1×1 (поточечная)

В каждой точке (n_1, n_2) вычисляет скалярное произведение по каналам — проекция вектора признака в новое пространство. Аналог однослойного перцептрона. Используется для изменения глубины тензора без изменения пространственного размера.

4.3 Слои СНС

Сверточный слой (CONV)

Основной слой. L фильтров, каждый $M_1 \times M_2 \times K$. Шаг (stride) управляет субдискретизацией.

Слой пулинга (POOL)

Уменьшает пространственный размер. Обучаемых параметров нет.

- **Max-pooling:** берёт максимум в локальном окне. Обеспечивает инвариантность к малым сдвигам.
- **Average-pooling:** усредняет значения в окне.
- **Global Max/Average Pooling:** берёт максимум/среднее по всей карте → одно число на карту.

Flatten

Разворачивает 3D-тензор в одномерный вектор для подачи в полносвязные слои.

Полносвязный слой (FC)

Классический перцептронный слой. Применяется в конце СНС для классификации.

4.4 Типовая архитектура СНС

$$INPUT \rightarrow [CONV \rightarrow RELU \rightarrow POOL] \times M \rightarrow FC \rightarrow SOFTMAX$$

Чередование свёрток с ReLU и пулингов, затем Flatten и полносвязные слои, завершаясь softmax для классификации.

По мере углубления: пространственный размер уменьшается, число каналов (фильтров) увеличивается. Это позволяет детектировать всё более сложные паттерны при сохранении объёма вычислений.

4.5 Поле восприятия (Receptive Field)

Область входа, влияющая на конкретный нейрон. С глубиной накопленное поле восприятия растёт. Два последовательных ядра 3×3 дают эффективное поле 5×5 , но требуют меньше параметров и добавляют нелинейность.

4.6 Специализированные слои

Транспонированная свёртка (Transposed Convolution)

«Обратная» по эффекту обычной свёртке: увеличивает пространственный размер тензора (upsampling). Применяется в декодерах, генеративных и сегментационных сетях.

Дырявая свёртка (Dilated / Atrous Convolution)

Вставляет промежутки между отсчётами ядра (коэффициент дырявости d). Увеличивает рецептивное поле при том же числе параметров без потери разрешения.

Разделяемые по глубине свёртки (Depthwise Separable)

Факторизует свёртку на: depthwise (отдельно по каждому каналу) + pointwise 1×1 . Резко снижает число параметров и вычислений (Xception, MobileNet).

Bottleneck (бутылочное горлышко)

Сжимает представление свёрткой 1×1 до меньшего размера, выполняет основную свёртку 3×3 , затем разворачивает обратно. Снижает вычислительные затраты.

4.7 Ключевые архитектуры

- **LeNet (1998)**: первая классическая CNN. Свёртки + пулинг + FC.
- **AlexNet (2012)**: прорыв на ImageNet. ReLU, Dropout, GPU-обучение.
- **VGGNet (2014–2015)**: однородная структура из малых ядер 3×3 . Блоки: несколько свёрток + пулинг, число фильтров удваивается при переходе между блоками.
- **GoogLeNet / Inception (2014+)**: inception-модули — параллельные ветви (1×1 , 3×3 , 5×5 , пулинг). Свёртки 1×1 снижают размерность.
- **ResNet (2015)**: остаточные (skip) соединения: $y = F(x) + x$. Решает проблему затухания градиента, позволяет обучать очень глубокие сети.
- **DenseNet (2016)**: конкатенация выходов всех предыдущих слоёв блока. Более гладкая поверхность потерь, лучше чем ResNet.
- **U-Net (2015)**: кодер-декодер с skip-связями между симметричными уровнями. Сегментация изображений.
- **YOLO (You Only Look Once)**: быстрый однопроходный детектор объектов.

4.8 Transfer Learning и Fine-tuning

Берут сеть, обученную на большом наборе данных (например, ImageNet), и дообучают (fine-tune) под целевую задачу. Особенно полезно при малом числе обучающих данных.

4.9 Враждебные атаки (Adversarial Attacks)

Незаметное изменение изображения, из-за которого классификатор ошибается, хотя исходное изображение распознавалось верно (пример: панда → гиббон с вероятностью 99.3%).

Объяснение: изображение находится в пространстве миллионной размерности; разделяющая граница может проходить вблизи корректного примера.

Классификация атак:

- По знанию: на белый ящик (известна модель, параметры, градиенты) и чёрный ящик (только вход/выход)
- По цели: целевые (в указанный класс) и нецелевые (любая ошибка)
- По методу: основанные на запросах и основанные на переносе

Тема 5. Рекуррентные нейронные сети (РНС / RNN)

5.1 Что такое РНС

РНС — нейронная сеть с обратными связями. Отображает последовательности входов в последовательности выходов. Ключевое отличие: скрытое состояние $h(t)$ несёт информацию о прошлом, обновляясь на каждом такте.

Разделение весов по времени: одни и те же параметры на каждом шаге — постоянный размер модели независимо от длины последовательности.

5.2 Vanilla RNN (простейшая РНС)

$$\begin{aligned}h(t) &= f_h(W_{hh} \cdot h(t-1) + W_{xh} \cdot x(t) + b_h) \\ y(t) &= f_y(W_{hy} \cdot h(t) + b_y)\end{aligned}$$

Матрицы весов:

- **W_{xh}** : вход → состояние
- **W_{hh}** : предыдущее состояние → текущее состояние
- **W_{hy}** : состояние → выход

Активации: f_h — $\tanh$ (типично), f_y — softmax (токены), sigmoid или тождественная.

5.3 Режимы работы

- **Many-to-many**: последовательность → последовательность (перевод, генерация)
- **Many-to-one**: последовательность → один выход (классификация текста)
- **One-to-many**: один вход → последовательность (описание изображения)

5.4 Обучение: BPTT

BPTT (Back-Propagation Through Time) — разворачивание сети по времени и применение обычного backprop. Градиенты вычисляются рекурсивно: вклад шага t зависит от вклада шага $t+1$. Сложность — линейная по числу тактов.

5.5 Проблемы градиентов в РНС

Градиент по времени содержит множители вида $(f'_h \cdot W_{hh})$. При прохождении через T шагов этот множитель возводится в степень T .

- **Спектральная норма $W_{hh} < 1$** : градиент затухает экспоненциально → сеть «забывает» давнее прошлое
- **Спектральная норма $W_{hh} > 1$** : градиент взрывается → нестабильное обучение

Спектральная норма оператора = максимальному сингулярному числу (= модулю наибольшего собственного значения для нормального оператора).

Методы борьбы:

- **Gradient Clipping**: ограничение нормы градиента сверху. Простой эвристический приём против взрыва.
- **Архитектурные решения**: LSTM/GRU с аддитивными обновлениями состояния (аналог skip-соединений).

5.6 Преимущества и недостатки простых РНС

Преимущества:

- Обработка входа любой длины
- Постоянный размер модели
- Разделение весов по времени

Недостатки:

- Медленность (вычисления строго последовательны — нельзя параллелить)
- Затухание информации о давнем прошлом
- Проблемы градиентов
- Нет прямого доступа ко всем элементам входа (в отличие от внимания)

5.7 GRU (Gated Recurrent Unit)

Управляемые рекуррентные блоки с двумя вентилями. Идея: аддитивное обновление состояния, аналог skip-соединений ResNet.

Два вентиля:

- **Вентиль сброса $r(t)$** : управляет краткосрочной памятью. $r(t) = \sigma(W_r \cdot [h(t-1), x(t)])$
- **Вентиль обновления $z(t)$** : управляет долгосрочной памятью. $z(t) = \sigma(W_z \cdot [h(t-1), x(t)])$

Кандидат нового состояния:

$$\tilde{h}(t) = \tanh(W_h \cdot [r(t) \circ h(t-1), x(t)])$$

Финальное состояние:

$$h(t) = z(t) \circ h(t-1) + (1-z(t)) \circ \tilde{h}(t)$$

При $z(t) = 1$: прошлое состояние передаётся без изменений (долгосрочная память). При $r(t) \rightarrow 0$: блок работает как перцептрон от текущего входа. Свойство: $1-\sigma(x) = \sigma(-x)$.

5.8 LSTM (Long Short-Term Memory)

Предложена Хохрайтером и Шмидхубером в 1997 г. Три вентиля + отдельная ячейка памяти $c(t)$. GRU (2014) — более простой вариант.

Три вентиля (все вычисляются через сигмоиду):

- **Входной вентиль $i(t)$** : что помещать в ячейку
- **Вентиль забывания $f(t)$** : когда сбрасывать ячейку
- **Выходной вентиль $o(t)$** : что читать из ячейки

Кандидат новой ячейки (через $\tanh$):

$$\tilde{c}(t) = \tanh(W_c \cdot [h(t-1), x(t)])$$

Обновление ячейки памяти:

$$c(t) = f(t) \circ c(t-1) + i(t) \circ \tilde{c}(t)$$

При $f=1, i=0$: информация сохраняется неопределённо долго — долгосрочная память.

Скрытое состояние / выход:

$$h(t) = o(t) \circ \tanh(c(t))$$

$c(t)$ — долгосрочная память, $h(t)$ — краткосрочная. Отсюда название: Long Short-Term Memory. LSTM не гарантирует полное отсутствие проблем градиента, но существенно облегчает передачу информации во времени.

$\circ$ — произведение Адамара (поэлементное умножение). Вентили используют сигмоиду (значения в $(0,1)$). Кандидат — через $\tanh$ (значения в $(-1,1)$, стабилизирует динамику).

GRU: 2 вентиля. LSTM: 3 вентиля + ячейка $c(t)$.

5.9 Генерация последовательностей

- **Жадный алгоритм (Greedy):** на каждом шаге берёт самый вероятный токен (argmax). Просто, но не оптимально глобально.
- **Beam Search:** хранит k лучших частичных гипотез, расширяя их. Лучший результат, чем жадный поиск.
- **Температурная выборка:** масштабирует логиты перед softmax. Высокая $T \rightarrow$ разнообразнее. Низкая $T \rightarrow$ детерминированнее (при $T \rightarrow 0 \approx$ жадный поиск).
- **Top-k / Top-p sampling:** исключает маловероятные токены. Снижает шанс бессмысленных продолжений.

Тема 6. Трансформеры

6.1 Предпосылки и определение

Трансформер предложен в 2017 г. в статье «Attention is all you need» авторами из Google Brain и Google Research. Ключевое преимущество над РНС: все позиции обрабатываются параллельно, и через механизм внимания напрямую связываются любые два токена. Трансформер — архитектура из слоёв, сочетающих механизм самовнимания (self-attention) и полносвязные слои.

6.2 Scaled Dot-Product Attention (SDPA)

Базовый строительный блок трансформера. Принимает на вход запросы Q, ключи K и значения V:

$$SDPA(Q, K, V) = SoftMax(K \cdot Q^T / \sqrt{d}) \cdot V$$

Результат — взвешенная комбинация значений V, где веса определяются близостью запросов к ключам.

Компоненты:

- **Q (Queries) — запросы:** то, что ищем
- **K (Keys) — ключи:** метки элементов, с которыми сравниваем
- **V (Values) — значения:** то, что извлекаем при совпадении

Деление на $\sqrt{d}$ (масштабирование): удерживает аргументы softmax в разумном диапазоне, предотвращает насыщение softmax и затухание градиентов.

Свойства SDPA:

- Нет обучаемых параметров (параметры добавляет МНА через проекции)
- Инвариантность к перестановке пар (K, V): перестановка (K,V) не меняет результат
- Эквивариантность по Q: перестановка строк Q → та же перестановка строк результата

6.3 Multi-Head Attention (МНА)

Расширение SDPA с обучаемыми параметрами. h «голов» — каждая проецирует Q, K, V своими матрицами в подпространство меньшей размерности:

$$\begin{aligned} head_i &= SDPA(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V) \\ MHA(Q, K, V) &= [head_1 ; \dots ; head_h] \cdot W^O \end{aligned}$$

Параметры оригинального трансформера: h = 8 голов, d = 512, d_K = d_V = 64 (d_K · h = d). Благодаря соотношению d_K · h = d общее число параметров МНА не зависит от числа голов. Разные головы улавливают разные аспекты данных (синтаксис, семантику и т.д.). Результат МНА при m запросах имеет размерность m × d.

6.4 Позиционное кодирование

Механизм внимания инвариантен к порядку токенов (SDPA). Поэтому к эмбедингам добавляются позиционные коды, несущие информацию о позиции:

$$x_i = x_i \cdot \sin + p_i$$

В оригинальной работе — синусоидальные функции разных частот (R = 10000):

$$p_{\{i, 2k\}} = \sin(i / R^{\{2k/d\}}), \quad p_{\{i, 2k+1\}} = \cos(i / R^{\{2k/d\}})$$

Синусоидальные коды работают для последовательностей произвольной длины и помогают модели обобщаться на длины, не встречавшиеся при обучении. Альтернатива — обучаемые позиционные эмбединги (GPT, BERT).

6.5 Позиционная сеть прямого распространения (PW-FFN)

Двухслойная полносвязная сеть, применяемая независимо к каждой позиции (с общими весами):

$$FFN(x) = W_2 \cdot \text{ReLU}(W_1 \cdot x + b_1) + b_2$$

В оригинале: $d = 512$, $d_{ff} = 2048$ ($= 4 \cdot d$). «Позиционная» — потому что обрабатывает каждую позицию отдельно.

6.6 Блок кодера трансформера

Кодер состоит из $N = 6$ одинаковых по структуре блоков. Каждый блок:

- Подслой 1: MHA (самовнимание: $Q = K = V$ = входная последовательность)
- Остаточное соединение + Layer Normalization: $\text{LN}(x + \text{MHA}(x))$
- Подслой 2: PW-FFN
- Остаточное соединение + LN: $\text{LN}(x + \text{FFN}(x))$

Кодер отображает n входных d -мерных векторов в n выходных представлений ($e_1 \dots e_n$) той же размерности, обогащённых контекстом.

6.7 Маскированное внимание (Masked MHA)

В декодере при предсказании t -го токена нельзя использовать будущие токены.

Маскирование технически реализуется добавлением $-\infty$ к запрещённым позициям перед $\text{softmax} \rightarrow$ их веса после $\text{softmax} = 0$.

6.8 Блок декодера трансформера

Декодер тоже $N = 6$ блоков, три подслоя в каждом:

- Masked MHA ($Q=K=V$ — выходная последовательность, маскирование будущего)
- Cross-Attention MHA: Q — из декодера, K и V — из кодера (матрица E)
- PW-FFN

Все с остаточными связями и LN. Ключевое: ключи и значения берутся из кодера — именно так декодер «смотрит» на исходную последовательность.

6.9 Режимы обучения и инференса

Инференс (генерация)

Декодер работает авторегрессионно: на каждом шаге предсказывает следующий токен, добавляет его ко входу, и так до токена EOS (End of Sequence). BinMax — argmax по вероятностям $\rightarrow$ one-hot вектор.

Обучение (Teacher Forcing)

Всей верной выходной последовательности подаётся сразу (со сдвигом на 1).

Маскированное внимание не даёт «подглядывать» вперёд. Это ускоряет обучение по сравнению с пошаговой генерацией.

Генерация стартует с токена SOS (Start of Sequence), заканчивается при предсказании EOS.

6.10 Pre-LN vs Post-LN

- **Post-LN (оригинал 2017):** LN применяется после агрегации: $\text{LN}(x + F(x))$
- **Pre-LN:** нормализация перед подблоком: $x + F(\text{LN}(x))$. Часто стабильнее при обучении.

6.11 Оценка качества: перплексия

$$PPL = \exp(-\frac{1}{T} \cdot \sum \log P(x_t | x_1, \dots, x_{t-1}))$$

Меньшая перплексия = лучше предсказывает. Модель с $PPL = k$ ведёт себя как равномерный выбор из k равновероятных токенов.

6.12 Модели на основе трансформеров

BERT

Использует только кодер. Двухнаправленный контекст (внимание ко всем токенам сразу, без маскирования). Предобучение: MLM (предсказание замаскированных токенов) + NSP (Next Sentence Prediction). Fine-tuning под конкретные задачи.

GPT

Использует только декодер (без кросс-внимания к кодеру). Предсказание следующего токена (однонаправленное). Двухэтапный подход: предобучение (большие неразмеченные тексты) + fine-tuning. Маскированное самовнимание.

Сравнение BERT vs GPT

- **BERT:** кодер, двухнаправленный контекст, подходит для понимания текста
- **GPT:** декодер, однонаправленный, подходит для генерации текста

6.13 Параметры базового трансформера

По расчётам методички: кодер ~31.7 млн параметров, декодер ~38 млн, итого ~68 млн обучаемых параметров в базовом трансформере.

6.14 Преимущества трансформеров перед РНС

- Параллельность: все позиции обрабатываются одновременно
- Прямой доступ к любым токенам через внимание (нет затухания по времени)
- Лучшее улавливание дальних зависимостей
- Масштабируемость: обучение на огромных данных

Шпаргалка: ключевые формулы и факты

Активационные функции

- **Сигмоида:** $f = 1/(1+e^{-s})$, $f' = f(1-f)$, диапазон $(0,1)$
- **tanh:** $f' = 1-\tanh^2$, диапазон $(-1,1)$
- **ReLU:** $f = \max(0,s)$, $f' = 1$ при $s>0$
- **Leaky ReLU:** $f = 0.01s$ при $s<0$

Backprop

- **Выходной слой:** $g = y(1-y)(d-y)$ для сигмоиды
- **Скрытый слой:** $g = y(1-y) \cdot \sum g_{\text{next}} \cdot w_{\text{next}}$
- **Обновление:** $w += \lambda \cdot g \cdot y_{\text{prev}}$
- **Softmax+CE:** производная = $y_k - 1$ (верный класс), y_k (иные)

Оптимизаторы

- **SGD:** $w -= \lambda \cdot g$
- **Momentum:** $v = \beta \cdot v - \lambda \cdot g$; $w += v$
- **Adam:** $m = \beta_1 m + (1-\beta_1)g$; $v = \beta_2 v + (1-\beta_2)g^2$; $w -= \lambda \cdot \hat{m} / \sqrt{\hat{v}}$

СНС

- **Valid-свёртка:** $(N-M+1) \times (N-M+1)$
- **Параметры фильтра:** $M_1 \cdot M_2 \cdot K + 1$
- **Нет параметров:** слой пулинга
- **VGG:** блоки 3×3 , число фильтров $\times 2$, размер $/2$
- **ResNet:** $y = F(x) + x$ (skip-соединения)
- **DenseNet:** конкатенация всех предыдущих слоёв

GRU и LSTM

- **GRU:** 2 вентиля (r, z); $h(t) = z \circ h(t-1) + (1-z) \circ \tilde{h}(t)$
- **LSTM:** 3 вентиля (i, f, o) + ячейка $c(t)$
- $c(t) = f \circ c(t-1) + i \circ \tilde{c}(t)$; $h(t) = o \circ \tanh(c(t))$
- **$c(t)$ — долгая память, $h(t)$ — краткая**

Трансформер

- **SDPA:** $\text{SoftMax}(KQ^T/\sqrt{d}) \cdot V$
- **MHA:** h голов, $d_K \cdot h = d$; $N=6$ блоков
- **FFN:** $W_2 \cdot \text{ReLU}(W_1 x + b_1) + b_2$, $d_{\text{ff}}=2048=4d$
- **Позиц. код:** $\sin/\cos$ разных частот + к эмбедингу
- **Маска:** $-\infty$ перед softmax для будущих позиций
- **BERT:** кодер, двунаправленный, MLM+NSP
- **GPT:** декодер, однонаправленный, предсказание следующего токена
- **Параметры:** ~68 млн в базовом трансформере